

# Data-Driven Cloth for Interactive Applications

Tejas Ponnappa (PES1UG24CS498) | Mukesh Sheregar (PES1UG25CS828)  
UE24CS352A Machine Learning – Mini-Project, 2026

## 1. Problem statement

Physics-based cloth simulation is accurate but far too slow for interactive uses such as games, e-commerce try-on or VR. Our assigned problem is to reproduce the data-driven approach of Xue & Wu (*Data-Driven Clothing for Interactive Applications*, Stanford CS229, 2021): learn a fast model that takes a character's pose and predicts how the clothing deforms. The model outputs the *offset* of every garment vertex from a rest garment (3 numbers per vertex). We compare a neural network that predicts PCA coefficients of these offsets against a linear-regression baseline and a network that predicts the offsets directly.

## 2. Dataset

**Original data.** The paper uses 10,000 random poses of a coat (2,170 vertices), skinned in Blender and simulated with PhysBAM, with 187 faulty simulations removed. That dataset and simulator are not public, so we used a public substitute.

**Our data: TailorNet.** TailorNet (Patel et al., CVPR 2020) is a public dataset of simulated garments worn by SMPL bodies. We use its *male short-pants* garment (2,710 vertices, so 8,130 outputs). From the 40 body-shape / garment-style combinations in the download we sampled up to 100 poses each: 3,833 poses. For every pose the target is the simulated garment minus the rest garment of that shape and style, converted to cm (mean offset 1.98 cm per vertex, maximum 38.6 cm). As in the paper, we remove implausible examples with a distance threshold (largest vertex offset above median + 5 robust standard deviations = 19.8 cm), which removed 91 poses, and split the remaining 3,742 at random into 2,245 train / 748 validation / 749 test. PCA is fitted on the training set only. The input has 86 numbers: the SMPL pose (72) plus body shape (10) and garment style (4), because poses from different bodies and styles are mixed. This differs from the paper (14 joint quaternions, one garment).

## 3. Approach

- **PCA on offsets.** Offsets are centred and projected onto the top- $k$  principal components (SVD of the training offsets):  $x \approx \sum_1 c_i u_i + \mu$ . This shrinks the output ( $8,130 \rightarrow k$ ).
- **Network.** A fully-connected network  $86 \rightarrow 128 \rightarrow 256$  (ReLU)  $\rightarrow k$  PCA coefficients, followed by a fixed (non-trainable) linear “PC layer” that reconstructs the 8,130 offsets, so the loss is the MSE on offsets:  $L = (1/n) \sum (x_i - p_i)^2$ , with  $p_i$  the predicted offset. Adam, learning rate  $1e-4$ , batch size 32, best-validation checkpoint, seed 0.
- **Baseline.** Linear regression (a network with no hidden layers,  $86 \rightarrow 8,130$ ), solved in closed form by least squares.
- **Ablation.** The same network predicting offsets directly (no PCA) and with  $k = 256, 128$  and 64 components, each trained for 200 and for 1,000 epochs (the paper's setting).

## 4. Implementation overview

The code is in PyTorch/NumPy/Matplotlib in a private GitHub repository (see its README for setup and the dataset download). `src/data_tailornet.py` loads TailorNet, builds the offsets, filters outliers and splits the data; `src/pca.py` fits PCA on training offsets; `src/model.py` defines the MLP with the fixed PCA layer and the closed-form baseline; `src/train.py` trains one model; `src/evaluate.py` computes test metrics, timings and plots; `demo.py` is a live 3D viewer showing the ground-truth and predicted garment for a chosen test pose. `run_all_tn.bat` runs the whole pipeline.

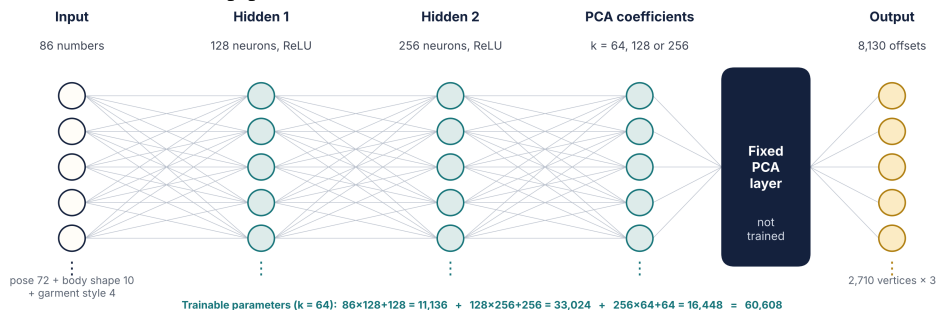


Figure 1: The network. 86 input numbers, two hidden ReLU layers (128 and 256 neurons),  $k$  PCA coefficients, and the fixed PCA layer that gives the 8,130 vertex offsets.

## 5. Results

Metrics are on the 749 held-out test poses with the best-validation checkpoint (single run). Test MSE is the mean squared error over all vertex coordinates; vertex error is the mean Euclidean distance per vertex.

| Model               | Output dim | Trainable params | Test MSE, 200 ep (cm <sup>2</sup> ) | Test MSE, 1000 ep (cm <sup>2</sup> ) | Mean vertex err., 1000 ep (cm) |
|---------------------|------------|------------------|-------------------------------------|--------------------------------------|--------------------------------|
| Predict train mean  | —          | 0                | 1.868                               | 1.868                                | —                              |
| Linear regression   | 8,130      | 707,310          | 0.907                               | 0.907                                | 1.207                          |
| MLP, direct offsets | 8,130      | 2,133,570        | 0.475                               | <b>0.379</b>                         | 0.753                          |
| MLP + PCA, k = 256  | 256        | 109,952          | 0.662                               | 0.403                                | 0.788                          |
| MLP + PCA, k = 128  | 128        | 77,056           | 0.662                               | 0.406                                | 0.793                          |
| MLP + PCA, k = 64   | 64         | 60,608           | 0.660                               | 0.410                                | 0.799                          |

Table 1: Test results. Predicting the training mean gives MSE 1.868. Linear regression is exact, so it is the same in both columns.

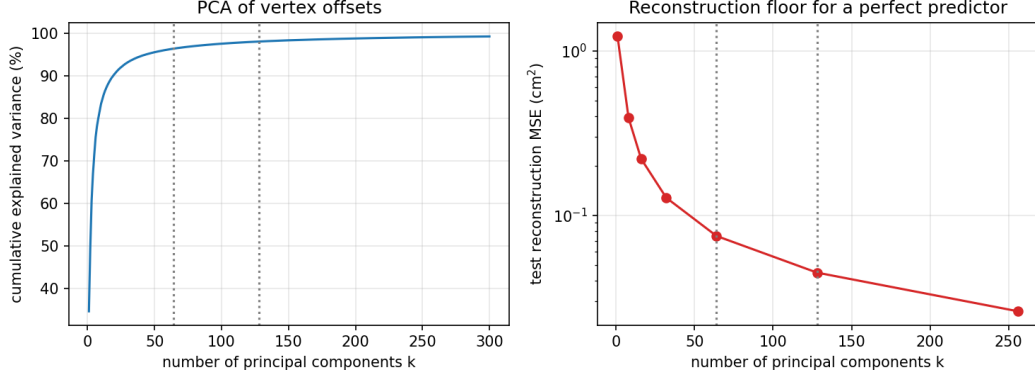


Figure 2: Cumulative explained variance of the offsets (left) and test reconstruction error of a perfect predictor (right) vs number of principal components  $k$ ; dotted lines mark  $k = 64$  and  $128$ .

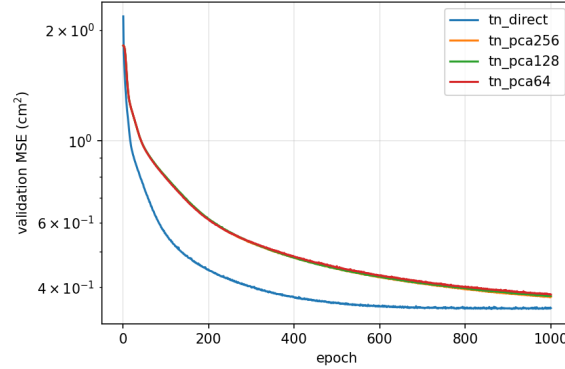


Figure 3: Validation MSE during 1,000 epochs (log scale). The direct network has levelled off; the three PCA networks (nearly overlapping) are still slowly improving.

## 6. Conclusions

- **Learned models clearly beat linear regression.** Test MSE falls from 0.907 to 0.379–0.410 (55–58% lower) and the mean vertex error from 1.21 to 0.75–0.80 cm. The best network is 2.4× better than linear regression (the paper reports 2.6×), so pose-to-cloth deformation is not captured well by a linear model.
- **PCA gives a much smaller network at a small cost in accuracy.** With  $k = 64$  the network has 60.6k trainable parameters versus 2.13M for direct prediction (35× fewer; 19× fewer for  $k = 256$ ), and its test MSE is 8% higher (0.410 vs 0.379). The paper found PCA about 6% better than direct prediction; we did not reproduce that. The direct network had levelled off (best epoch 909), whereas the PCA networks were still improving at epoch 1,000 and the gap shrank from 39% at 200 epochs to 6–8% at 1,000, so longer training might close it (not tested).
- **Number of components.** More components were slightly better (0.403, 0.406, 0.410 for  $k = 256, 128, 64$ ), but the differences are under 2%, within the noise of a single run, so we cannot name a best  $k$  (the paper found 128). The reconstruction error of a perfect predictor is about 0.075, 0.045 and 0.026 for  $k = 64, 128, 256$ , well below the model error, so PCA itself is not what limits accuracy.
- **Speed.** On a CPU, predicting a batch of 256 poses takes 0.015–0.064 ms per pose and a single pose 0.3–2.1 ms (single-pose timings were noisy), far below a 16 ms frame budget.
- **Limitations / future work:** one garment, one random seed and 3.7k poses (the paper used 10k); the random split puts test poses on the same body shapes and styles as training, so unseen bodies are not tested; PCA models were not fully converged. Future work: several seeds, longer training, and a split by body shape.

**References:** K. Xue and J. Wu, “Data-Driven Clothing for Interactive Applications”, Stanford CS229 project report, Spring 2021 ([cs229.stanford.edu/proj2021spr/report2/81970649.pdf](https://cs229.stanford.edu/proj2021spr/report2/81970649.pdf)). C. Patel, Z. Liao and G. Pons-Moll, “TailorNet: Predicting Clothing in 3D as a Function of Human Pose, Shape and Garment Style”, CVPR 2020.